

VADEMECUM NA KOŁOKWIUM (JAVA)

Skondensowane kompendium wzorców kodu (tzw. "ściągałka") zebranych z kolokwii z programowania na przestrzeni lat 2021-2024. Obejmuje tworzenie komunikacji sieciowej (Sockets), interfejsów (JavaFX), zrównoleganie obliczeń, REST API (Spring Boot) oraz łączenie z bazą danych (SQLite JDBC).

1. Połączenia Sieciowe (Gniazda / Sockets)

Wielowątkowy serwer i odbieranie tekstu

```
import java.net.ServerSocket;
import java.net.Socket;
import java.io.*;

public class ServerApp {
    public static void main(String[] args) throws IOException {
        try (ServerSocket server = new ServerSocket(5000)) {
            while (true) {
                Socket client = server.accept(); // Blokuje do połączenia
                new Thread(() -> handleClient(client)).start();
            }
        }

        private static void handleClient(Socket socket) {
            try (BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
                PrintWriter out = new PrintWriter(socket.getOutputStream(), true)) {

                String line;
                while ((line = in.readLine()) != null) {
                    System.out.println("Otrzymano: " + line);
                    out.println("Echo: " + line);
                }
            } catch (IOException e) { e.printStackTrace(); }
        }
    }
}
```

Przesyłanie plików binarnych (np. obraz PNG)

```
// ODBIÓR PLIKU (DataInputStream)
DataInputStream dis = new DataInputStream(socket.getInputStream());
long fileSize = dis.readLong();
try (FileOutputStream fos = new FileOutputStream("output.png")) {
    byte[] buffer = new byte[8192];
    long received = 0;
    int count;
    while (received < fileSize) {
        count = dis.read(buffer, 0, (int) Math.min(buffer.length, fileSize - received));
        if (count == -1) break;
        fos.write(buffer, 0, count);
        received += count;
    }
}
```

2. Bazy Danych (SQLite / JDBC)

```
import java.sql.*;

// Połączenie i stworzenie tabeli z autoinkrementacją (DML)
String url = "jdbc:sqlite:baza.db";
try (Connection conn = DriverManager.getConnection(url);
     Statement stmt = conn.createStatement()) {
    stmt.execute("CREATE TABLE IF NOT EXISTS wpisy (" +
        "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
        "tekst TEXT, x INTEGER, y INTEGER)");
} catch (SQLException e) { e.printStackTrace(); }

// Wstawianie danych do bazy (PreparedStatement chroni przed SQL Injection)
String insertSQL = "INSERT INTO wpisy (tekst, x, y) VALUES (?, ?, ?)";
try (Connection conn = DriverManager.getConnection(url);
     PreparedStatement pstmt = conn.prepareStatement(insertSQL)) {
    pstmt.setString(1, "Przykładowy tekst");
    pstmt.setInt(2, 10);
    pstmt.setInt(3, 20);
    pstmt.executeUpdate();
}

// Odczytywanie danych
try (Connection conn = DriverManager.getConnection(url);
     Statement stmt = conn.createStatement();
     ResultSet rs = stmt.executeQuery("SELECT tekst, x, y FROM wpisy ORDER BY id")) {
    while (rs.next()) {
        String txt = rs.getString("tekst");
        int x = rs.getInt("x");
    }
}
```

3. Interfejs Użytkownika (JavaFX)

Aktualizowanie widoku z innego wątku (Niezbędne!)

```
// Zmiany w UI wywoływane przez serwer / inny wątek MUSZĄ być w runLater
Platform.runLater(() -> {
    label.setText("Nowy status serwera");
});
```

Rysowanie (Canvas) i eventy klawiatury

```
Canvas canvas = new Canvas(500, 500);
GraphicsContext gc = canvas.getGraphicsContext2D();

// Czyszczenie i rysowanie linii
gc.setFill(Color.WHITE);
gc.fillRect(0, 0, 500, 500);
gc.setStroke(Color.RED);
gc.setLineWidth(2.0);
gc.strokeLine(x1, y1, x2, y2);

// Rejestrowanie wciśnień (Scene musi mieć focus)
scene.setOnKeyPressed(event -> {
    switch (event.getCode()) {
        case LEFT: x -= 10; break;
        case RIGHT: x += 10; break;
    }
});
```

Filtrowanie i Sortowanie (ListView)

```
ObservableList<String> data = FXCollections.observableArrayList();
FilteredList<String> filtered = new FilteredList<>(data, p -> true);

textField.textProperty().addListener((obs, oldV, newV) -> {
    filtered.setPredicate(item -> {
        if (newV == null || newV.isEmpty()) return true;
        return item.startsWith(newV);
    });
});

// Sortowanie listy po przefiltrowaniu
SortedList<String> sorted = new SortedList<>(filtered, Comparator.naturalOrder());
listView.setItems(sorted);
```

4. Zrównoleglanie Zadań (ExecutorService)

```
// Pobranie ilości rdzeni dla utworzenia puli wątków
int cores = Runtime.getRuntime().availableProcessors();
ExecutorService executor = Executors.newFixedThreadPool(cores);

for (int i = 0; i < cores; i++) {
    final int threadId = i;
    executor.submit(() -> {
        // Logika realizowana w wielu wątkach równolegle...
        System.out.println("Wątek: " + threadId);
    });
}
executor.shutdown(); // Konieczne zamknięcie puli (nie przerywa już dodanych)
try {
    executor.awaitTermination(1, TimeUnit.HOURS); // Czekanie na zakończenie
} catch (InterruptedException e) { e.printStackTrace(); }
```

5. Przetwarzanie Obrazu i FFMPEG (ProcessBuilder)

```
// Odczyt i zapis obrazu modyfikując piksele
BufferedImage img = ImageIO.read(new File("input.png"));
int argb = img.getRGB(x, y);
img.setRGB(x, y, Color.RED.getRGB());
ImageIO.write(img, "png", new File("output.png"));

// Odpalenie procesu zewnętrznego i zablokowanie do końca
ProcessBuilder pb = new ProcessBuilder(
    "ffmpeg", "-framerate", "30", "-pattern_type", "glob",
    "-i", "*.png", "-c:v", "libx265", "out.mp4"
);
pb.directory(new File("workdir")); // Katalog roboczy
pb.inheritIO(); // Zrzuca logi procesu do naszej konsoli
Process p = pb.start();
p.waitFor();
```

6. Spring Boot (REST API i Kontrolery)

```
import org.springframework.web.bind.annotation.*;
import org.springframework.http.ResponseEntity;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;

// Klasa rekordu DTO dla zapytania JSON, np: {"id":"abc", "x":5}
public record PixelDTO(String id, int x) {}

@RestController
public class ApiController {

    @GetMapping("/tokens")
    public List<String> getTokens() {
        return List.of("Token1", "Token2"); // Automatycznie parsuje do JSON (GET)
    }

    @PostMapping("/pixel")
    public ResponseEntity<Void> setPixel(@RequestBody PixelDTO request) {
        if (request.x() < 0) {
            return new ResponseEntity<>(HttpStatus.BAD_REQUEST); // Błąd 400
        }
        return new ResponseEntity<>(HttpStatus.OK); // Sukces 200
    }

    @GetMapping(value = "/image", produces = MediaType.IMAGE_PNG_VALUE)
    public byte[] getImage() throws Exception {
        // Zwracanie pliku binarnego / renderowanego obrazka
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        ImageIO.write(canvasImage, "png", baos);
        return baos.toByteArray();
    }
}
```